</> codigo.py

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 6s >

Este arquivo é malware (não é benigno).

Ele corresponde exatamente a variantes do malware **GlassWorm** (e campanhas similares documentadas em 2026) que usam a blockchain Solana como canal de comando e controle (C2) descentralizado, baixam Node.js e executam payload ofuscado/criptografado via `eval`. Relatórios de segurança (Annex Security, Yeeth Security, OpenSource Malware, The Force Multiplier etc.) descrevem o mesmo endereço Solana, as mesmas funções ofuscadas (`_getSignFAddress`, `fvzsrnzlh`, `scyzzvvy`), o arquivo `init.json`, o download de Node.js v22.9.0 e a execução de JS com `atob` + `eval`.

Análise passo a passo

1. **Funções ofuscadas e nomes sem sentido**
   Nomes como `fvzsrnzlh`, `scyzzvvy`, `uezupbxi`, `ypbtv`, `zmwtjkgxor` etc. são típicos de ofuscação de malware. Não há lógica legítima que justifique isso.
2. **Consulta à blockchain Solana como C2**
   - Chama `getSignaturesForAddress` no endereço fixo `BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC` em vários endpoints RPC públicos.
   - Procura transações que contenham **memo**.
   - Extrai o memo, remove o prefixo `[número]`, faz `json.loads` e obtém um campo `link` (URL em base64).
     Isso é um "dead-drop" clássico: o atacante atualiza o C2 publicando uma nova memo on-chain (impossível de tirar do ar facilmente). O endereço e o padrão são os mesmos dos relatórios de GlassWorm.
3. **Anti-análise / geo-bloqueio**
   - Função `_isRussianSystem()` verifica idioma, locale, fuso horário e offset UTC.
   - Se detectar sistema russo, **não executa**.
     Isso é típico de malware de origem russa (ou que evita alvos russos).
4. **Persistência e rate-limit**
   - Cria/usa `~/init.json` com um timestamp.
   - Se o arquivo existir e tiver menos de 2 dias, pula a execução.
   - Em macOS força a criação do arquivo.
     Serve para não re-executar constantemente e para marcar infecção.
5. **Download silencioso de Node.js**

- Baixa Node.js v22.9.0 oficial (Windows/macOS/Linux, x64/ARM) para o diretório home do usuário.
- Extrai e usa o binário local.
  Motivo: o payload final (infostealer) é escrito em JavaScript; o Python só serve de dropper multiplataforma.

6. **Download do payload e execução**
   - Decodifica a URL do memo (`base64.b64decode(data['link'])`).
   - Faz GET na URL, lendo também headers ofuscados (`ivbase64` e `secretkey` decodificados de base64).
   - Monta um arquivo temporário `i.js` com:

     ```JavaScript
     var https = require("https");
     const secretKey = '...';
     const _iv = Buffer.from('...', "base64");
     eval(atob('...'));   // payload criptografado/ofuscado
     ```

   - Executa via `subprocess.Popen([node_exe, js_file_path], ...)` **escondendo stdout/stderr** e (no Windows) com `CREATE_NO_WINDOW` .
     O payload final (após AES-256-CBC + `eval`) é tipicamente um **infostealer** focado em:
   - Carteiras de criptomoedas (MetaMask, Phantom, etc.)
   - Seed phrases / chaves privadas
   - Credenciais de navegador, SSH, tokens de desenvolvedor, etc.
7. **Outros comportamentos suspeitos**
   - Atraso inicial de 10 segundos.
   - Loop de polling a cada 10 segundos até encontrar memo.
   - Tratamento de erros que tenta novamente.
   - Emulação de `Buffer` do Node.js dentro do Python.
   - Nenhum propósito legítimo (não é ferramenta de monitoramento, não é script de usuário, não é biblioteca útil).

Conclusão

O código é um **dropper/loader de malware** que:

- Usa Solana como C2 resistente a takedown;
- Evita sistemas russos;
- Baixa e executa Node.js + payload JS ofuscado/criptografado;
- Tem como objetivo final o roubo de dados sensíveis (principalmente cripto e credenciais de desenvolvedores).

Não execute este arquivo. Se ele já rodou na sua máquina, considere o sistema comprometido: troque senhas/chaves de carteiras a partir de outro dispositivo limpo, verifique `~/init.json`, instalações inesperadas de Node.js e extensões/pacotes suspeitos.

Qual é a categoria principal deste malware?

Worked for 10s >

Categoria principal: Dropper / Loader (com payload final de Infostealer).

Detalhamento:

Aspecto	Classificação
Este arquivo específico (Python)	**Dropper / Loader**
Função principal	Baixa Node.js + payload JS ofuscado/criptografado e o executa
Payload final	**Infostealer** (roubo de carteiras de criptomoedas, seed phrases, chaves privadas, credenciais de navegador, SSH keys, tokens etc.)
Técnica de C2	Blockchain (Solana memo) — típico de campanhas modernas de stealer
Família associada	GlassWorm (e variantes similares)

Por que não é outra categoria?

- **Não é ransomware** → não criptografa arquivos nem pede resgate.
- **Não é worm clássico** → não se propaga sozinho pela rede (a propagação ocorre via supply-chain: pacotes npm/PyPI ou extensões VS Code comprometidas).
- **Não é backdoor puro** → o objetivo final não é manter acesso remoto genérico, e sim roubar dados sensíveis.
- **Não é rootkit** → não tenta se esconder em nível de kernel.

Resumo:

Este script é um **dropper/loader** multiplataforma cuja missão é entregar e executar um **infostealer** (principalmente focado em criptomoedas e credenciais de desenvolvedores).

↘ Qual é a categoria do payload final?

↘ Como funciona a técnica de C2 via blockchain?

↘ Como detectar este malware